

Lecture # 12

SQLite Database

By

Dr. Muhammad Bilal

SQLite Database in Android

- Structure Query Language(SQL) is a database query language used for storing and managing data in Relational Database Management System (DBMS)
- This lecture assumes that you are familiar with SQL databases
- The APIs you'll need to use a database on Android are available in the “android.database.sqlite”

Define a schema and contract

- One of the main principles of SQL databases is the schema: a formal declaration of how the database is organized.
- The schema is reflected in the SQL statements that you use to create your database.
- You may find it helpful to create a companion class, known as a *contract* class, which explicitly specifies the layout of your schema in a systematic and self-documenting way.

Contract Class in SQLite

- A contract class is a container for constants that define names for tables, and columns.
- The contract class allows you to use the same constants across all the other classes in the same package.
- This lets you change a column name in one place and have it propagate throughout your code.
- A good way to organize a contract class is to put definitions that are global to your whole database in the root level of the class.

Contract Class for School (SchoolContract.java)

```
public final class SchoolContract {

    public static class Student {
        // Tables and Column names
        public static final String TABLE_NAME = "student";
        public static final String COLUMN_ID = "id";
        public static final String COLUMN_NAME = "name";
        public static final String COLUMN_ADDRESS = "address";

        // SQL queries
        public static final String CREATE_STUDENT_TABLE =
            "CREATE TABLE " + TABLE_NAME + "(" +
                COLUMN_ID + " INTEGER PRIMARY KEY, " +
                COLUMN_NAME + " TEXT, " +
                COLUMN_ADDRESS + " TEXT) ";

        public static final String DELETE_STUDENT_TABLE =
            "DROP TABLE IF EXISTS " + TABLE_NAME;

        public static final String READ_ALL_TABLE = "SELECT * FROM " + TABLE_NAME;
    }
}
```

SQLite Helper Class

- Once you have defined how your database looks, you should implement methods that create and maintain the database and tables using the “SQLiteOpenHelper” class
- The “SQLiteOpenHelper” class contains a useful set of APIs for managing your database.
- The database can be manage by calling “getWritableDatabase()” or “getReadableDatabase()” callbacks in helper class
- When you extends your helper class with “SQLiteOpenHelper” two callbacks must be implemented
 - onCreate()
 - onUpgrade()

SQLite Helper Class (SchoolDBHelper.java)

```
public class SchoolDBHelper extends SQLiteOpenHelper {  
  
    public SchoolDBHelper(@Nullable Context context) {  
        super(context, DATABASE_NAME, null, DATABASE_VERSION);  
    }  
  
    @Override  
    public void onCreate(SQLiteDatabase db) {  
  
    }  
  
    @Override  
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {  
  
    }  
}
```

SQLite Helper Class Methods

```
public SchoolDBHelper(@Nullable Context context)
```

- This is the constructor class to specify the database name and its version

```
public void onCreate(SQLiteDatabase db)
```

- This method is called when database is created, “SQLiteDatabase” has methods to create, delete, and execute SQL commands, e.g. “execSQL” executes SQL query

```
public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion)
```

- Called when the database is updated using old and new database version number

SQLite Helper Class (SchoolDBHelper.java)

```
public class SchoolDBHelper extends SQLiteOpenHelper {
    Context context;
    public static final int DATABASE_VERSION = 1;
    public static final String DATABASE_NAME = "School.db";

    public SchoolDBHelper(@Nullable Context context) {
        super(context, DATABASE_NAME, null, DATABASE_VERSION);
        this.context = context;
    }

    @Override
    public void onCreate(SQLiteDatabase db) {
        db.execSQL(SchoolContract.Student.CREATE_STUDENT_TABLE);
    }

    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
        // This database is only a cache for online data, so its upgrade policy is
        // to simply to discard the data and start over
        db.execSQL(SchoolContract.Student.DELETE_STUDENT_TABLE);
        onCreate(db);
    }
}
```

Insert Entries in SQLite DB

- Once the database has been created, the next step is to populate the database with the entries
- Let assume a android app for School Information System, where a database for student is entered and displayed in list view



“ContentValues” for Entires in SQLite

- **ContentValues** is a maplike class that matches a value to a String key
- It contains multiple overloaded put methods that enforce type safety
- Here is a list of the put methods supported by **ContentValues**
 - void put(String key, String value)
 - void put(String key, Integer value)
 - String getString(String Key)
 - Integer getInt(String Key)
 - Integer getColumnIndex(String ColumnName)
- Now we will create another method for inserting entries in database in SQLite helper class

```
// add id, name, address to table  
public void addEntry(int id, String name, String address)
```

Insert Entries using insert()

- The insert() first argument is Table, and last is values to be inserted
- The Second argument is to specify what to do is values are empty, is specify null then no values will be inserted

```
// add id, name, address to table
public void addEntry(int id, String name, String address){
    SQLiteDatabase db = getWritableDatabase();
    // Create a new map of values, where column names are the keys
    ContentValues values = new ContentValues();
    values.put(SchoolContract.Student.COLUMN_ID, id);
    values.put(SchoolContract.Student.COLUMN_NAME, name);
    values.put(SchoolContract.Student.COLUMN_ADDRESS, address);
    try{
        // Insert the new row
        db.insert(SchoolContract.Student.TABLE_NAME, null, values);
    } catch (Exception e){
        Toast.makeText(context, "Error: " + e, Toast.LENGTH_SHORT).show();
    }
    // must close the db
    db.close();
}
```

Read All Entries from SQLite Table

- The SQL query can be implemented to read all entries from table by executing the “rawQuery()” method, which will return the “Cursor”
- Cursors are what contain the result set of a query made against a database in Android
- The Cursor class has an API that allows an app to read (in a type-safe manner) the columns that were returned from the query as well as iterate over the rows of the result set
- Implement another method “readAll()” in SQLite helper class to read all entries from table and display to display it on list view

“readAll()” in SQLite Helper Class

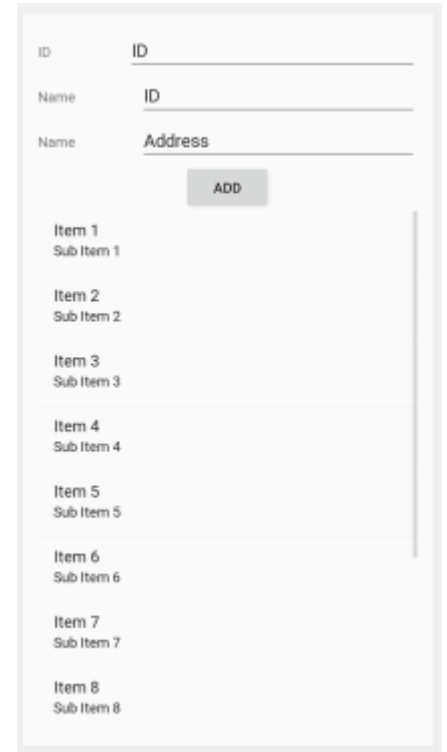
- “cursor.moveToNext()” will return true and move to next row until not all return rows are finished reading
- The return rows are inserted

```
public ArrayList<SchoolSetter> readAll(){
    SQLiteDatabase db = getWritableDatabase();
    Cursor cursor = db.rawQuery(SchoolContract.Student.READ_ALL_TABLE,null);

    ArrayList<SchoolSetter> setters = new ArrayList<SchoolSetter>();
    while (cursor.moveToNext()){
        setters.add(new SchoolSetter(
            cursor.getInt(cursor.getColumnIndex(SchoolContract.Student.COLUMN_ID)),
            cursor.getString(cursor.getColumnIndex(SchoolContract.Student.COLUMN_NAME)),
            cursor.getString(cursor.getColumnIndex(SchoolContract.Student.COLUMN_ADDRESS))
        ));
    }
    return setters;
}
```

Main Activity XML Layout (activity_main.xml)

- Design a main activity with three edit text box for student ID, name, and address
- Using the ADD button the entries in the edit text boxes inserted into the database
- The list view will display the all entries in the database



The image shows a mobile application UI design. At the top, there are three input fields: 'ID', 'Name', and 'Address'. Each field has a label to its left and a corresponding input line. Below the 'Address' field is a grey 'ADD' button. Underneath the button is a list view containing eight items. Each item consists of a main title (Item 1 through Item 8) and a subtitle (Sub Item 1 through Sub Item 8). The list is enclosed in a light grey border with rounded corners.

MainActivity.java

```
public class MainActivity extends AppCompatActivity {
    EditText idET, nameET, addressET;
    SchoolDBHelper dbHelper;
    ListView listView;
    SchoolBaseAdapter adapter;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        idET = findViewById(R.id.id_edit_box);
        nameET = findViewById(R.id.name_edit_box);
        addressET = findViewById(R.id.addr_edit_box);

        listView = findViewById(R.id.dbListView);
        // create DB
        dbHelper = new SchoolDBHelper(getApplicationContext());

        ArrayList<SchoolSetter> schoolSetters = dbHelper.readAll();
        adapter = new SchoolBaseAdapter(getApplicationContext(), schoolSetters);

        listView.setAdapter(adapter);
    }

    public void onUpdateDB(View view) {
        int id = Integer.valueOf(idET.getText().toString());
        String name = nameET.getText().toString();
        String address = addressET.getText().toString();
        dbHelper.addEntry(id, name, address);
        updateListView();
    }

    void updateListView() {
        ArrayList<SchoolSetter> schoolSetters = dbHelper.readAll();
        adapter = new SchoolBaseAdapter(getApplicationContext(), schoolSetters);
        listView.setAdapter(adapter);
        listView.invalidateViews();
    }
}
```


SchoolBaseAdapter.java

```
public class SchoolBaseAdapter extends BaseAdapter {
    ArrayList<SchoolSetter> setter;
    LayoutInflater inflater;
    public SchoolBaseAdapter(Context context, ArrayList<SchoolSetter> setter) {
        this.setter = setter;
        inflater = LayoutInflater.from(context);
    }

    @Override
    public int getCount() {
        return setter.size();
    }

    @Override
    public Object getItem(int position) {
        return setter.get(position);
    }

    @Override
    public long getItemId(int position) {
        return position;
    }

    @Override
    public View getView(int position, View convertView, ViewGroup parent) {
        // get view of custom item layout
        convertView = inflater.inflate(R.layout.list_view_item, null);
        TextView id = convertView.findViewById(R.id.item_id);
        TextView name = convertView.findViewById(R.id.item_name);
        TextView address = convertView.findViewById(R.id.item_address);

        id.setText(String.valueOf(setter.get(position).getId()));
        name.setText(String.valueOf(setter.get(position).getName()));
        address.setText(String.valueOf(setter.get(position).getAddress()));
        return convertView;
    }
}
```

SchoolSetter.java

```
public class SchoolSetter {  
    int id;  
    String name;  
    String address;  
  
    public SchoolSetter(int id, String name, String address) {  
        this.id = id;  
        this.name = name;  
        this.address = address;  
    }  
  
    public int getId() {  
        return id;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public String getAddress() {  
        return address;  
    }  
}
```

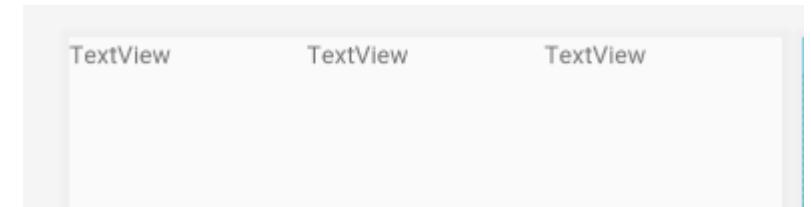
List View Item Layout (list_view_item.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="horizontal"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:id="@+id/item_id"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:text="TextView" />

    <TextView
        android:id="@+id/item_name"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:text="TextView" />

    <TextView
        android:id="@+id/item_address"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:text="TextView" />
</LinearLayout>
```



SchoolContract.java

```
public final class SchoolContract {

    public static class Student {
        // Tables and Column names
        public static final String TABLE_NAME = "student";
        public static final String COLUMN_ID = "id";
        public static final String COLUMN_NAME = "name";
        public static final String COLUMN_ADDRESS = "address";

        // SQL queries
        public static final String CREATE_STUDENT_TABLE =
            "CREATE TABLE " + TABLE_NAME + "(" +
                COLUMN_ID + " INTEGER PRIMARY KEY, " +
                COLUMN_NAME + " TEXT, " +
                COLUMN_ADDRESS + " TEXT) ";

        public static final String DELETE_STUDENT_TABLE =
            "DROP TABLE IF EXISTS " + TABLE_NAME;

        public static final String READ_ALL_TABLE = "SELECT * FROM " + TABLE_NAME;
    }
}
```

SchoolDBHelper.java

```
public class SchoolDBHelper extends SQLiteOpenHelper {
    Context context;
    public static final int DATABASE_VERSION = 1;
    public static final String DATABASE_NAME = "School.db";

    public SchoolDBHelper(@Nullable Context context) {
        super(context, DATABASE_NAME, null, DATABASE_VERSION);
        this.context = context;
    }

    @Override
    public void onCreate(SQLiteDatabase db) {
        db.execSQL(SchoolContract.Student.CREATE_STUDENT_TABLE);
    }

    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
        // This database is only a cache for online data, so its upgrade policy is
        // to simply to discard the data and start over
        db.execSQL(SchoolContract.Student.DELETE_STUDENT_TABLE);
        onCreate(db);
    }

    // add id, name, address to table
    public void addEntry(int id, String name, String address){
        SQLiteDatabase db = getWritableDatabase();
        // Create a new map of values, where column names are the keys
        ContentValues values = new ContentValues();
        values.put(SchoolContract.Student.COLUMN_ID, id);
        values.put(SchoolContract.Student.COLUMN_NAME, name);
        values.put(SchoolContract.Student.COLUMN_ADDRESS, address);
        try{
            // Insert the new row
            db.insert(SchoolContract.Student.TABLE_NAME, null, values);
        } catch (Exception e){
            Toast.makeText(context, "Error: " + e, Toast.LENGTH_SHORT).show();
        }
        db.close();
    }

    public ArrayList<SchoolSetter> readAll(){
        SQLiteDatabase db = getWritableDatabase();
        Cursor cursor = db.rawQuery(SchoolContract.Student.READ_ALL_TABLE, null);

        ArrayList<SchoolSetter> setters = new ArrayList<SchoolSetter>();
        while (cursor.moveToNext()){
            setters.add(new SchoolSetter(
                cursor.getInt(cursor.getColumnIndex(SchoolContract.Student.COLUMN_ID)),
                cursor.getString(cursor.getColumnIndex(SchoolContract.Student.COLUMN_NAME)),
                cursor.getString(cursor.getColumnIndex(SchoolContract.Student.COLUMN_ADDRESS))
            ));
        }
        return setters;
    }
}
```